A Quantitative Characterization of Semantic Conflicts in Open-Source Java Projects

Anonymous Author(s)

***Abstract*—Collaborative software development relies on automated merge tools, but parallel contributions can introduce dynamic semantic conflicts that alter program behavior without triggering textual merge conflicts. Different techniques can be used to detect these interferences, including testing, dynamic analysis, and static analysis. However, the prevalence, structural characteristics, and distribution of semantic conflicts remain largely unexplored at scale. To address this gap, we establish a comprehensive catalog of topological layouts that define the structural characteristics of semantic conflicts. We evaluate this catalog using interprocedural static analysis on a large benchmark comprising 907 real-world Java merge scenarios from open-source GitHub repositories. Our results reveal a stark structural dichotomy: while conflicts caused by converging data flows concentrate overwhelmingly in fully local topologies (90.4% start and end in the same file), conflicts caused by direct data flows or overridden assignments exhibit highly divergent behavior, with multi-file topologies accounting for 60.3% and 46.7% of their respective instances. Furthermore, unlike state override conflicts which distribute sparsely, data-flow interferences cluster heavily in specific problematic merge scenarios. These findings provide actionable guidance for software integration pipelines, suggesting tiered detection strategies that rely on fast intraprocedural checks for local confluences while reserving deeper, multi-file traversal specifically for divergent data-flow and override patterns.**

***Index Terms*—Semantic Conflict, Static Analysis, Merge Tools, Direct Flow (DF), Confluence Flow (CF), Override Assignment (OA)**

I. Introduction

Collaborative software development depends on developers continuously merging their contributions into shared repositories. Modern version control systems automate most of this integration through text-based merge tools such as `diff3` [1], which detect and report syntactic clashes when developers modify the same or consecutive lines of code. Despite the maturity of these textual merge algorithms, a class of subtle integration errors evades them entirely: *dynamic semantic conflicts* [2], [3] occur when parallel modifications from two branches, each individually correct, produce unintended behavior in the merged program, which is undetected by conventional text-based merge tools.

Previous studies have explored three categories of semantic conflicts that capture common interference patterns [4]. A *Direct Flow* (DF) conflict arises when one branch defines or modifies a state element and the other branch reads that state element, breaking the implicit data flow assumptions of both developers. A *Confluence Flow* (CF) conflict occurs when two branches independently alter separate state elements that jointly influence a third, dependent state element, thereby distorting the merged computation. An *Override Assignment* (OA) conflict happens when both branches write to the same state element with no intervening base write between them, so the merge silently discards one developer's intent. Because none of these interferences triggers a textual merge conflict, they easily slip past code review and corrupt production behavior.

To detect semantic conflicts before they reach production, researchers have developed static analysis tools that traverse program call graphs to identify DF, CF, and OA interferences [4], [5]. These tools operate on three-way merge scenarios and track state element definitions and uses across branches. However, while these analyses are able to detect semantic conflicts in real-world projects, a fundamental gap remains: the *prevalence*, *structural characteristics*, and *topological distribution* of each specific conflict type have not been quantitatively characterized at scale. Without this empirical knowledge, tool designers lack the data needed to calibrate analysis depth, balance detection recall against computational cost, and prioritize the most impactful conflict patterns.

Understanding the structural characteristics and prevalence of semantic conflicts is important for three reasons. First, recognizing the most recurrent patterns helps developers and integrators anticipate and avoid risky merge scenarios. Second, empirical knowledge of these conflicts allows researchers to direct future studies more effectively, exploring novel alternatives for conflict resolution. Third, similar to how catalogues of build conflicts enable the development of program repair and awareness tools [6], a topological classification of semantic conflicts can guide the creation of automated repair techniques and proactive warnings in modern development environments. Ultimately, this foundational knowledge is essential to optimize future static analyses.

To address this gap, we conduct a comprehensive quantitative characterization of DF, CF, and OA semantic conflicts. First, we establish a complete catalog of 54 reference oracle layouts that define the possible topologies of these interferences based on file boundaries and call-graph shape. We then run interprocedural static analysis on the RefDataset [7], a large benchmark comprising 907 real-world Java merge scenarios from open-source GitHub repositories. For each detected interference, we extract the originating and terminating nodes of the conflicting execution path, abstract chained calls within the same file, and match the resulting structure against our catalog. We then quantify the frequency of each layout and analyze locality, symmetry, and call-graph depth distributions across all detected conflicts.

Our results reveal two key findings. First, semantic con-

flicts present a stark structural dichotomy: while CF conflicts concentrate overwhelmingly in fully local topologies (90.4% belong to layouts that start and end in the same file), both DF and OA conflicts exhibit highly divergent behavior, with topologies that start in the same file but end in different files accounting for 60.3% of the 31,747 DF instances and 46.7% of the 105 OA instances. Second, conflict occurrences exhibit contrasting distributions across merge scenarios: while OA interferences appear sparsely across many merges, DF and CF conflicts are highly concentrated in a small subset of problematic merges, with a single architectural merge generating thousands of DF instances. These findings provide actionable guidance for configuring static analysis tools in daily software integration pipelines.

Contributions. This paper makes the following contributions:

1) A comprehensive catalog of topological layouts that define how DF, CF, and OA semantic conflicts manifest across file boundaries and call-graph structures.
2) A quantitative characterization of the prevalence, locality, symmetry, and depth distribution of 32,361 semantic conflict instances across 907 real-world merge scenarios.
3) Actionable guidance for configuring static analysis tools, balancing detection recall against computational cost in daily software integration pipelines.

II. Semantic Conflicts in Practice

To understand the behavioral interferences that this study characterizes, this section defines the three semantic conflict types under analysis, DF, CF, and OA, and illustrates each with a concrete code scenario.

To detect behavioral interferences that escape textual merge tools, researchers have formalized three categories of semantic conflicts based on how parallel modifications interact with shared program state [4]. Below we present the formal definitions of DF, CF, and OA conflicts, following the formalization established by Santos de Jesus et al. [4], alongside illustrative examples.

A. DF Conflicts

A DF conflict captures an interference between a *definition* and a *use* of the same program state element across two branches. Formally, let $S^B_{def}(v)$ denote the set of statements that define state element v in branch B, and $S^B_{use}(v)$ the set of statements that use v in branch B. A DF interference exists when:

$$\begin{aligned} &\exists\, v,\ s_L \in S^L_{def}(v) \setminus S^B_{def}(v),\ s_R \in S^R_{use}(v) \setminus S^B_{use}(v) \\ \text{or}\quad &\exists\, v,\ s_L \in S^L_{use}(v) \setminus S^B_{use}(v),\ s_R \in S^R_{def}(v) \setminus S^B_{def}(v) \end{aligned} \tag{1}$$

In other words, one branch introduces a new definition of v (not present in the base) while the other branch introduces a new use of v, or vice versa. The merged program then contains a direct flow path that neither developer anticipated, because the definer did not expect external reads, and the user did not expect the value to change.

Figure 1 (left card) illustrates the canonical DF pattern. In the base version, state element `x` holds the value `0`. The Left branch assigns `x = 42`, while the Right branch introduces a `print(x)` statement. After merge, the print statement reads the value `42`, a value that the Right-branch developer never anticipated. The merged program compiles and runs without errors, yet its behavior differs from what either developer intended.

Listing 1 presents a more realistic scenario involving a configuration class (in this and all subsequent code listings, lines modified by the Left branch are highlighted in pink, and lines modified by the Right branch are highlighted in green). Developer Left modifies a default timeout value, while Developer Right adds a new method that reads the same field to compute a retry interval. After the merge, the retry interval silently doubles because the timeout constant changed.

```
class RetryPolicy {
  int timeout = 30 60; // Left branch:
      doubled for slow networks
  int interval(Config c) {
    return c.timeout * 2; // Right
       branch: expects 60, gets 120
  }
}
```

Listing 1. DF conflict: Developer Left changes a timeout constant; Developer Right reads the same field to compute a retry interval.

B. CF Conflicts

A CF conflict captures an interference in which two branches independently modify *different* state elements that jointly influence a common dependent expression. Formally, a CF interference exists when:

$$\begin{aligned} \exists\, v_L \neq v_R,\ s_L \in S^L_{def}(v_L) \setminus S^B_{def}(v_L), \\ s_R \in S^R_{def}(v_R) \setminus S^B_{def}(v_R), \\ \exists\, e_B \in Expr^B : uses(e_B) \supseteq \{v_L, v_R\} \end{aligned} \tag{2}$$

The two modified state elements v_L and v_R flow into a base-version expression e_B, which now computes a result influenced by both changes simultaneously—a combined effect that neither developer anticipated individually.

Figure 1 (center card) illustrates the canonical CF pattern. In the base, both `x` and `y` equal `0`. Developer Left sets `x = 5`; Developer Right sets `y = 10`. After the merge, the expression `z = x + y` evaluates to `15`, a value that neither developer expected, since each assumed the other state element remained at its base value.

Listing 2 shows a practical CF scenario involving a pricing computation.

```
class Invoice {
  // Left branch: increases discount
  double discount = 0.10 0.20;
  // Right branch: increases tax
  double tax = 0.05 0.12;
```

```
  // Merged: finalPrice combines both
     changes
  double finalPrice(double base) {
    return base * (1 - discount) * (1 +
       tax);
  }
}
```

Listing 2. CF conflict: Developer Left changes the discount rate; Developer Right changes the tax rate; the final price reflects both changes simultaneously.

C. OA Conflicts

An OA conflict captures an interference in which both branches write to the *same* program state element, with no intervening base-version write. Formally, an OA interference exists when:

$$\exists v,\ s_L \in S^L_{def}(v) \setminus S^B_{def}(v),\ s_R \in S^R_{def}(v) \setminus S^B_{def}(v) \quad (3)$$

Both branches add new definitions of v that did not exist in the base version. After the merge, the program contains both writes, and the execution order determines which assignment "wins." One developer's intent is silently lost because their write is overridden by the other developer's assignment.

Figure 1 (right card) shows the canonical OA pattern. Both Developer Left and Developer Right write to state element `y`: Left assigns `y = 5`, Right assigns `y = 10`. After the merge, the program contains both assignments in sequence, and the second one silently overwrites the first.

Listing 3 demonstrates a practical OA scenario.

```
class Logger {
  // Left branch: changes default level
  Level level = ~~Level.INFO;~~ Level.DEBUG;

  void init() {
    loadConfig();
    // Right branch: overrides level
    level = Level.WARN;
  }
}
// Merged: init() assignment silently
   overwrites the default
```

Listing 3. OA conflict: both developers independently set the logging level; the merge silently discards one assignment.

III. Empirical Study Setup

This section describes the setup of our quantitative characterization study. We first formulate the research questions guiding our investigation. Next, we explain how we define the topological layouts that capture the structural patterns of semantic conflicts. We then detail the dataset and describe the study procedures, including our interprocedural static analysis approach and instrumentation. Figure 1 provides an overview of the empirical evaluation pipeline.

A. Research Questions

We formulate two research questions to guide our empirical characterization:

- RQ1 What are the possible topological patterns in which DF, CF, and OA semantic conflicts manifest, and what structural properties define each layout?
- RQ2 How many conflicts of each topological layout occur in real-world open-source Java merge scenarios?

RQ1 presents and explains the complete catalog of topological layouts for each conflict type, providing a visual and structural understanding of how semantic interferences materialize across files, methods, and call graphs. RQ2 quantifies the occurrence of each layout by executing the static analysis on the RefDataset, revealing which patterns dominate in practice and which remain rare.

B. Topological Layout Definition

Before executing the large-scale experiment, we established a foundational scientific contribution of this work: the systematic definition and cataloging of the possible topological structures for DF, CF, and OA conflicts. To derive these layouts, we employed a hybrid research method. First, in an exploratory phase, we observed the raw outputs of static analysis executions on a subset of merge scenarios to extract initial, empirically grounded conflict structures. Next, through structured group discussions, we generalized these initial cases by building upon interference graph formation rules, adjusting them to capture broader interference patterns. This rigorous process allowed us to systematically extrapolate and map all other structurally distinct permutations, aiming for a comprehensive coverage of how these semantic conflicts can manifest.

●	Modification of the Left branch
●	Modification of the Right branch
●	Base code (neutral context)
▭	File Boundary (Class)
⟶	Regular method call (or call chain)
- - -➤	Interference (breakdown of expectations)
1	Number of distinct files involved
A	Arbitrary identifiers

Fig. 2. Visual anatomy caption defining the structural components used in the topological layouts.

In our visual anatomy of these layouts (Figure 2), solid-border boxes represent distinct file boundaries (classes). Circular nodes represent specific lines of code, color-coded to indicate their different branches of origin: green circles for modifications of the Left branch, magenta circles for modifications of the Right branch, and gray circles for the base code (neutral context). Solid black arrows denote regular method

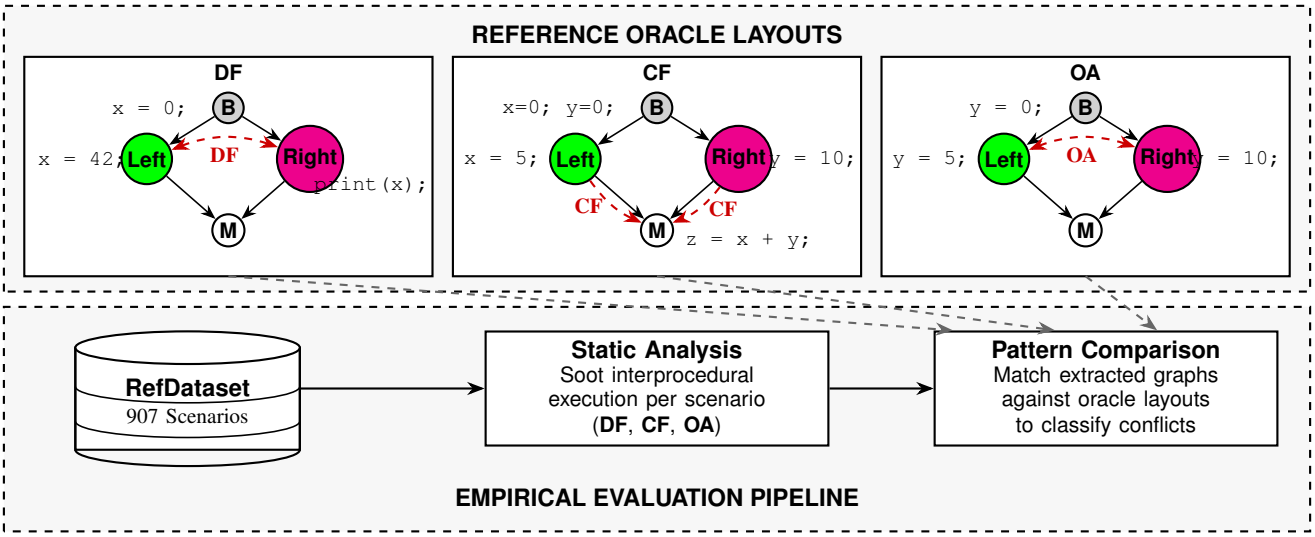

Fig. 1. Workflow of the empirical study.

calls (or call chains), while dashed red arrows explicitly mark the interferences (conflicts).

To formalize the creation and differentiation of the cataloged layouts, we established a specific set of structural rules:

1) **Call abstraction:** Calls to the same file and/or chained calls are abstracted into a single call arrow. This ensures that only the modified lines and the final lines of call sequences appear in the layout.
2) **Color symmetry:** The colors of the nodes can be alternated (maintaining the same set of nodes with equal colors) in a layout without changing its fundamental structure.
3) **Layout differentiation:** A layout A is considered structurally distinct from a layout B if at least one of the following conditions is true:
   (i) The number of files between A and B is different.
   (ii) The number of call arrows between A and B is different.
   (iii) The nodes involved in the conflict are located differently (in different files) between A and B.

The layouts are identified using an alphanumeric naming convention (e.g., A1, A2, B2, C3). In this notation, the numeric digit has a strict meaning: it indicates the exact number of distinct files involved in the conflict layout. Conversely, the letters (A, B, C, etc.) serve as arbitrary identifiers to distinguish the different structural permutations mapped and organized in our catalog.

Fundamentally, DF and OA layouts exhibit a binary structural symmetry—differing primarily in the direction of their dependency flow. In DF, the flow propagates from a state element's definition to its use; in OA, the relationship is inverted, originating from the overriding assignment towards the original definition. Conversely, CF layouts present a wider topological variance. Because CF involves the convergence of multiple distinct flows into a single common point, they admit a wide range of configurations, reflecting the higher inherent complexity of confluence points.

This analysis produced a comprehensive catalog of reference layouts, which we present as the first result in Section IV. To ensure that this empirically derived catalog is exhaustive and captures all structurally distinct permutations under our rules, we also present a mathematical combinatorial proof in Section IV-A3.

C. Dataset

To answer our research questions, we use the RefDataset [7][1]. This dataset comprises 907 experimental units from 295 open-source Java projects where two contributors modified the same method or constructor. We selected these units using the following criteria: GitHub repositories with at least 10 stars, using Maven or Gradle, and passing tests within 30 minutes on JDK 8, 11, or 17. An additional filter confirmed that the mutually modified target classes were included in the final `.jar` file.

For each scenario, we build the JAR file of the merge commit version without external dependencies. This reduces the amount of code analyzed, improving execution time, although it may decrease accuracy of the static analysis tooling; this trade-off is promising for practical uses of static analysis in detecting interference, and thus we adopt this design. Along with this, we collect the line numbers corresponding to the changes introduced by the Left and Right parent branches of the merged revision. By design, the chosen dataset restricts its scope to cases where two developers modified the same method, providing scenarios that are naturally more prone to behavioral interference.

D. Study Procedures and Instrumentation

We run the static analysis to detect DF, CF, OA conflicts for each experimental unit in the dataset. To detect

[1]https://github.com/spgroup/ref-dataset

interferences that span method boundaries, the analyses are executed interprocedurally using the Soot 4.5.0 program analysis framework [8] equipped with the SPARK pointer-analysis engine [9].

For each scenario, the analysis pipeline proceeds in four main steps. First, during bytecode loading, Soot loads the compiled JAR files for the merged versions. We execute our experiment with default settings for the Jimple Body Creation (`jb`) phase of Soot, except for the `use-original-names` option, which preserves the original names of local state elements. We also configure Soot to keep line numbers from bytecode, which is essential to precisely map interferences back to the source code.

Second, we leverage the SPARK framework to build a whole-program call graph using context-insensitive points-to analysis. To focus the analysis on developer-changed code, we configure Soot not to explore classes in standard Java libraries (e.g., `ArrayList` and `HashMap`), except for basic classes like `String`, `RuntimeException`, and wrapper classes for primitive types.

Third, during interference detection, the analysis starts from each changed statement in Left or Right and traverses the call graph by recursively expanding callees, accumulating state definitions and uses along the call chain. Recent work has investigated the impact of the analysis *depth limit* on semantic conflict detection, showing that while deeper limits (e.g., 10 levels) can capture additional interferences, they also trigger timeout explosions and increase computational costs with diminishing returns [10]. Thus, following prior methodologies, we configure a depth limit of 5 levels from the analysis entry point to provide an optimal balance between execution time and detection accuracy.

Finally, in the topological classification step, the static analysis outputs a detailed report of the detected interferences, which our procedure then uses to classify each conflict into a topological category. Instead of analyzing every step of the collision, our classification algorithm parses the execution stacks provided in the report, extracting only the origin and the outcome of the conflicting paths. Specifically, the algorithm crosses the reported execution stack with the modified lines, discarding the initial non-modified nodes so that the stack starts at the first modified line. Then, calls to the same file and/or chained calls are abstracted into a single instance, eliminating the redundancy of multiple consecutive calls. This abstraction ensures that only the modified lines and the last lines of the call sequences appear in the extracted topology. The exact size and shape of these two extracted file sequences determine the structural typology of the conflict, which is then matched against the reference oracle layouts defined in Section III-B.

To avoid variations in software configuration, we execute all experiments inside a single Docker container. The container runs on an Ubuntu machine equipped with an Intel Xeon Gold 6338 processor and 2 TB of RAM, ensuring consistent execution conditions and abundant computational resources.

We set the timeout to 300 seconds as the maximum time a user should reasonably wait for this type of analysis. If this time limit is exceeded, the analysis is interrupted.

IV. Results

This section presents the results organized by our two research questions. We first characterize the topological layouts for each conflict type (RQ1), then quantify the occurrence of each layout across the RefDataset (RQ2).

A. RQ1: Topological Layouts of Semantic Conflicts

We systematically derived the complete catalog of topological layouts by applying the structural differentiation rules defined in Section III-B to the formal conflict definitions from Section II. The derivation enumerates all structurally distinct arrangements of the conflict entities across file boundaries and call-graph edges.

We organize the catalog along two primary dimensions of scope. The first is origin scope: whether the conflict *starts in the same file* (i.e., both the Left and Right modifications originate in a single class) or in different files. The second is convergence scope: whether the conflict *ends in the same file* (i.e., the interference point resides in the same file as one of the origin modifications) or whether it diverges to a separate file entirely. Combining these two binary properties yields four distinct, mutually exclusive topological groups: (i) starts and ends in the same file, (ii) starts in the same file but ends in a different file, (iii) starts in different files but ends in one of them, and (iv) neither starts nor ends in the same file. Table I summarizes the resulting layout distributions across these four groups.

TABLE I
Topological layouts by origin and convergence scope.

Conflict Type	Starts & Ends Same File	Starts Same, Ends Diff.	Starts Diff., Ends Same	Neither (Scattered)	Total
DF and OA	2	2	1	6	11
CF	4	7	10	11	32

The figures that follow use the visual notation defined in Figure 2. In the following subsections we discuss representative layouts for each conflict type; the complete catalog, containing detailed descriptions and code examples for all remaining layouts, is available in our supplementary material [11].

1) DF and OA Layouts: DF and OA conflicts share a structurally identical catalog of 11 layouts (Table I). Both conflict types involve exactly two conflicting entities per branch; the only semantic difference is the direction of the dependency flow (definition-to-use for DF; write-to-write for OA). Because the topological classification depends solely on the number of files involved and the multi-file call-graph structure—not on the direction of dependency—the two catalogs are equivalent. Figure 3 presents all 11 layouts organized by the number of distinct files involved (rows), with columns serving as arbitrary identifiers for the different structural permutations.

As shown in Table I, the 11 layouts for DF and OA span all four topological groups. In the fully local group, **A1**

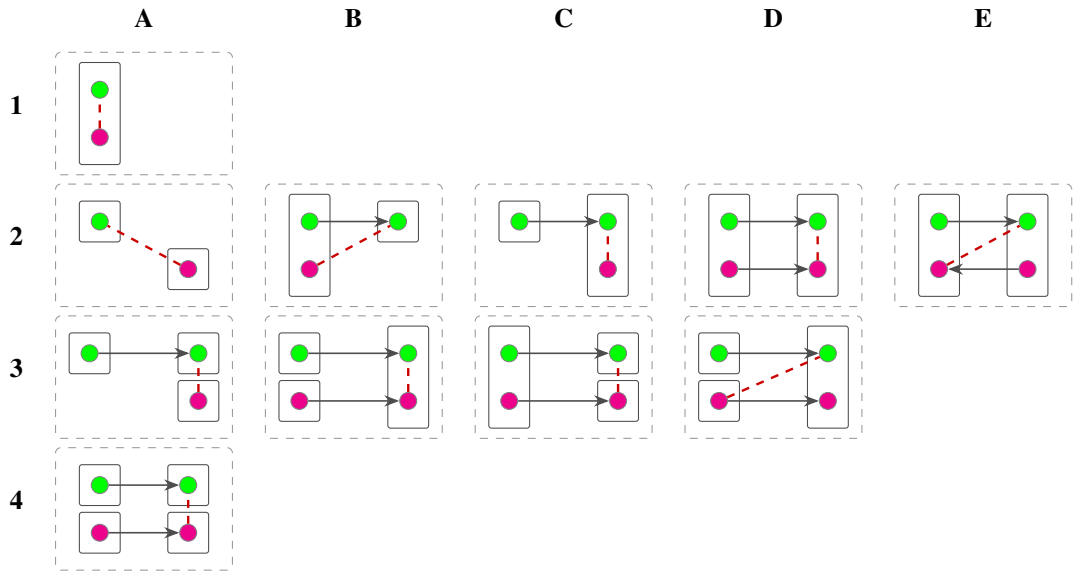

Fig. 3. Topological layouts shared by DF and OA conflicts.

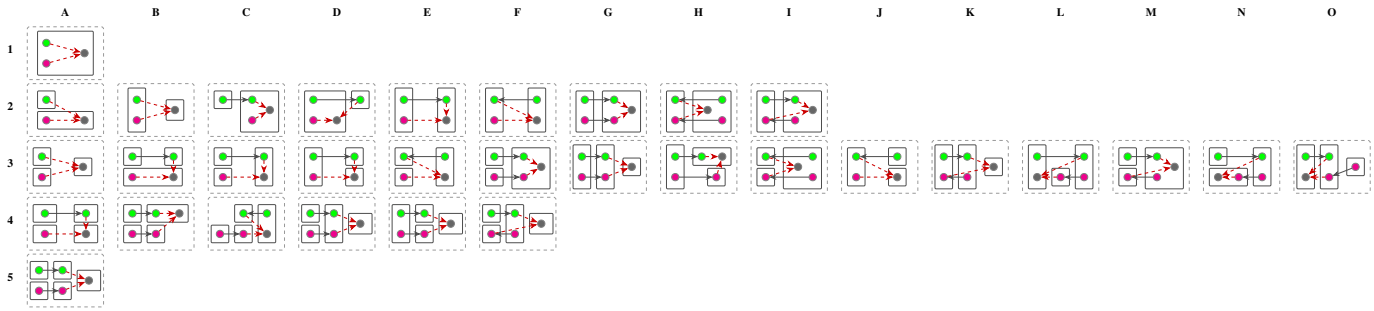

Fig. 4. Topological layouts for CF conflicts.

serves as the baseline configuration where all modifications and interferences occur within a single class, corresponding to the motivating examples in Listings 1 and 3. Divergent layouts, such as **B2**, originate locally but cross file boundaries, separating the interference points. Conversely, **C2** illustrates a convergent scenario where independent modifications propagate via a method call into a shared origin file. Finally, fully scattered conflicts like **A3** involve distinct files where the interference occurs independently across boundaries without any method calls between the modifications themselves. The full descriptions of the remaining seven layouts (D2, C3, A2, E2, B3, D3, A4) are available in our online appendix [11].

2) CF Layouts: CF conflicts involve three entities, the Left modification, the Right modification, and the base-version dependent expression, whose relative positions across file boundaries and call-graph edges determine the layout. This three-way dependency structure admits 32 distinct topological arrangements, compared to 11 for DF and OA (Table I). The differentiation conditions from Section III-B apply directly: layouts G2 and G3 are distinct due to condition (i); C2 and G2 are distinct due to condition (ii); and G2 and I2 are distinct due to condition (iii). Figure 4 presents these layouts organized by file count (rows), with columns serving as arbitrary identifiers for the different structural permutations. Among the 32 CF layouts, four are fully local (e.g., the baseline **A1**, corresponding to the motivating example in Listing 2), while the remaining 28 introduce varying degrees of scattering. For example, **B2** originates locally but converges in an external file, separating the origin from the convergence point. Conversely, **A2** originates across file boundaries but converges back into an origin file, meaning the interference happens locally for Right but remotely for Left. Finally, eleven layouts are fully scattered, such as **A3**, where independent modifications flow into a shared expression in a completely separate third class. The full descriptions of the remaining 28 layouts are available in our online appendix [11].

3) Theoretical Completeness of the Catalog: To verify the completeness of our empirically derived catalog, we model layout formation mathematically using graph partitioning and Bell numbers (B_n). Under our rules, any modification branch has at most its origin (O) and destination (E) nodes, with $O \neq E$ due to call abstraction (Rule 1).

For DF and OA conflicts, we partition four nodes (L_o, L_e, R_o, R_e) across file boundaries. Enforcing $L_o \neq L_e$ and $R_o \neq R_e$, and factoring out symmetric pairs via Burnside's Lemma (Rule 2), yields exactly 11 valid permutations.

For CF conflicts, partitioning a five-node set $\{L_o, L_e, R_o, R_e, Dep\}$ under the same constraints yields exactly 32 permutations. This combinatorial proof confirms our catalog is exhaustive.

RQ1

We derived a complete catalog of topological layouts for semantic conflicts: 11 layouts shared by DF and OA, and 32 layouts for CF. Organizing the catalog along origin and convergence scopes reveals a diverse distribution: while fully local conflicts (starts and ends in the same file) are structurally possible across all types, the vast majority of possible topological layouts exhibit some form of scattering, either originating in different files, converging in different files, or both.

B. RQ2: Quantification of Conflict Occurrences

Before presenting the occurrence data, it is crucial to note that our dataset is constructed exclusively from merge scenarios where both developers modified the exact same method (*mutually modified methods*). This design choice is a limitation of the current static analysis tool. Consequently, the observed distributions only reflect topological layouts where the conflicting edits originate from a single shared file. Topologies where the conflicting edits originate in distinct files (e.g., A2, A3, A4) are inherently excluded from our dataset by construction.

We analyzed the occurrence of each topological layout to understand which patterns dominate in practice. Figures 5, 6, and 7 present the distributions for DF, OA, and CF conflicts based on our static analysis. For clarity, we display only the most frequent topologies.

For DF conflicts, excluding unmodified stack traces and symmetric modified lines, we successfully classified 31,747 instances. A striking observation is that all 31,747 instances belong to layouts where the modifications *start in the same file* (Groups 1 and 2), reflecting the constraint of our empirical dataset which isolates conflicts originating from same-file modifications. Within this constraint, 60.3% of the occurrences belong to the *Starts Same, Ends Different* group (Group 2), driven by divergent topologies like **B2** (10,947 occurrences, 34.5%) and **C3** (8,181 occurrences, 25.8%). The fully local *Starts & Ends in the Same File* group (Group 1) accounts for the remaining 39.7%, distributed between **A1** (8,912 occurrences, 28.1%) and **D2** (3,707 occurrences, 11.7%).

For OA conflicts, excluding cases with symmetric modified lines and unmodified stack traces, we successfully classified 105 instances. Similar to DF, all instances originate from the same file due to dataset constraints. The fully local *Starts & Ends in the Same File* group slightly edges out the divergent group, accounting for 53.3% of occurrences distributed between **A1** (32 occurrences, 30.5%) and **D2** (24 occurrences, 22.9%). The *Starts Same, Ends Different* group accounts for the remaining 46.7%, distributed among **C3** (31 occurrences, 29.5%) and **B2** (18 occurrences, 17.1%).

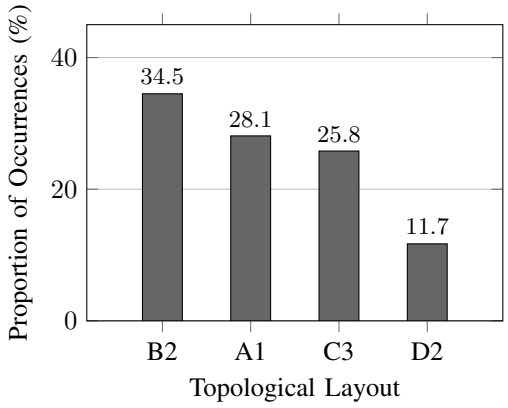

Fig. 5. Distribution of topological layouts for DF conflicts.

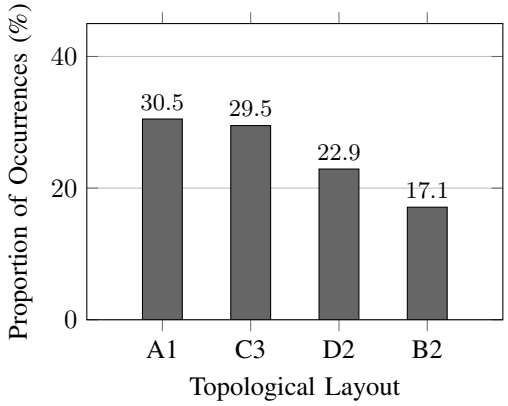

Fig. 6. Distribution of topological layouts for OA conflicts.

For CF conflicts, excluding unmodified stack traces and symmetric modified lines, we successfully classified 509 instances. Unlike DF and OA, CF conflicts are overwhelmingly local: 90.4% belong to the *Starts & Ends in the Same File* group, dominated almost entirely by the **A1** layout (458 occurrences, 90.0%), with a minor contribution from **E2** (2 occurrences, 0.4%). The *Starts Same, Ends Different* group constitutes a minority (9.6%) of the conflicts, with the most frequent divergent topologies being **B2** (27 occurrences, 5.3%) and **G2** (19 occurrences, 3.7%). Two additional divergent layouts appear at low rates: **D3** (2 occurrences, 0.4%) and **G3** (1 occurrence, 0.2%).

Layouts D3 and E2 were not discussed as representatives in Section IV-A; we define them here for completeness. **D3** involves Left propagating via a method call from a first file to a second file; Right occurs directly in the first (origin) file; and the base-code dependent expression is located in a third distinct file. **E2** is a two-file layout where Left propagates via a method call from a first file to a second file, Right occurs directly in the first file, and the base-code dependent expression is also in the first file.

This distribution highlights a fundamental dichotomy: while CF conflicts overwhelmingly start and end within a single

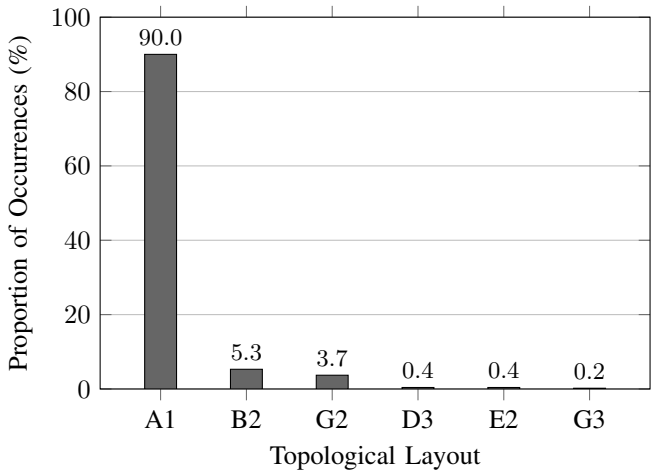

Fig. 7. Distribution of topological layouts for CF conflicts.

file (Group 1), both DF and OA conflicts frequently diverge, starting in a single file but concluding their interferences in separate classes (Group 2).

1) Conflict Distribution Across Merge Scenarios: We analyzed the distribution of conflicts across merge scenarios to determine whether they spread evenly or cluster in specific merges. OA conflicts distribute uniformly and sparsely. Of 213 evaluated scenarios, 82 contained valid OA conflicts (maximum four per scenario), with the top 10% of scenarios accounting for just 21.0% of occurrences. This indicates OA conflicts act as isolated events across many merges.

In contrast, CF and DF conflicts concentrate heavily in specific problematic merges. For CF, 139 of 234 scenarios contained instances, yet the top 10% account for 41.5% of occurrences, with one scenario generating 38 conflicts. This suggests localized structural changes affect multiple state elements simultaneously. DF conflicts exhibit extreme clustering: while 374 of 547 scenarios contained valid instances, the top 10% account for 77.9% of the 31,747 total occurrences. One single scenario generated 4,665 DF conflicts, and three others surpassed 2,000. This severe concentration implies that DF conflicts stem from broad architectural changes that introduce project-wide data-flow interference, rather than uniform, everyday integration noise.

RQ2

The empirical distribution reveals a structural dichotomy: CF conflicts occur primarily as fully local phenomena (Group 1), whereas DF and OA conflicts frequently diverge into different files (Group 2). Due to dataset limitations, all observed conflicts originated in the same file, but their convergence scopes vary significantly by conflict type.

V. Discussion and Study Implications

This section discusses the findings of our empirical characterization and outlines practical implications for software engineers, static analysis tool designers, and researchers based on the topological layouts and their occurrences.

A. Implications for Practitioners and Tool Configuration

Our results provide concrete guidance for configuring static analysis tools in real-world integration pipelines. The structural dichotomy between CF conflicts and DF/OA conflicts suggests that practitioners should adopt tiered detection strategies. For CF conflicts, an intraprocedural analysis (or a shallow interprocedural analysis that does not cross file boundaries) is sufficient to detect the vast majority of instances, as they are overwhelmingly concentrated in the fully local *Starts & Ends in the Same File* group (90.4%). Conversely, for DF and OA conflicts, interprocedural analysis is essential—a purely intraprocedural check would miss 60.3% of DF interferences and 46.7% of OA interferences, which belong to the divergent *Starts Same, Ends Different* group. Practitioners who integrate semantic conflict detection into CI/CD pipelines can use this distinction to run fast, shallow CF checks on every pull request while reserving deeper, multi-file DF and OA analysis for nightly or pre-release builds where more computational resources are available.

B. Implications for Tool Designers

The concentration of CF conflicts in fully local topologies suggests that tool designers can optimize analysis performance by prioritizing intraprocedural paths before expanding into multi-file call chains. A breadth-first traversal strategy—exhausting same-file callees before crossing file boundaries—would detect most CF interferences at minimal computational cost. Furthermore, knowing that both DF and OA conflicts frequently exhibit divergent multi-file patterns like B2, C3, and D2, tools must specifically target these distributed call chains when tracking data flows and state element overrides, optimizing the search space for the most common distributed interferences.

C. Implications for Researchers

The comprehensive catalog of 54 topological layouts (11 DF, 32 CF, 11 OA) presented in RQ1 provides a formal baseline for evaluating future semantic conflict detection techniques. Researchers can use these reference layouts to synthesize test cases that ensure a new analysis tool can correctly identify all possible structural variants, not just the fully local layout that starts and ends in the same file. The occurrence data from RQ2 highlights which layouts are most critical for practical applicability, demonstrating that benchmarks must heavily emphasize divergent topologies for DF and OA.

Our study focuses exclusively on Java merge scenarios. The high locality of CF conflicts (A1 dominance) and the distributed nature of DF and OA conflicts might generalize to other statically typed, object-oriented languages with similar scoping rules. However, languages with different

dispatch mechanisms (for example, Python's dynamic dispatch or JavaScript's prototype chains) or different modularity paradigms (such as C's header files) may exhibit entirely different topological distributions. Replicating this characterization study on multi-language benchmarks using our topological layout definitions constitutes a natural and necessary extension of our work.

VI. Threats to Validity

We organize the limitations and threats to the validity of this empirical characterization study following the four-category taxonomy by Wohlin et al. [12].

A. Construct Validity

Construct validity concerns whether our measurement metrics reflect the intended concepts. Our topological classification relies on extracting stack traces from static analysis reports to determine file boundary crossings and structural relationships. While this approach accurately captures the structural properties of interferences detected by the analysis, it may miss semantically equivalent interferences that operate through complex cross-method aliasing patterns not fully resolved by SPARK's context-insensitive points-to analysis. Furthermore, SPARK operates under an open-world assumption that may trim unreachable call edges, potentially excluding valid multi-file call paths in projects that rely heavily on dependency injection or runtime class loading. Additionally, Soot and SPARK possess known limitations in resolving Java reflection, dynamic proxies, and lambda expressions, which may result in missed interferences inside code paths that utilize these features.

B. Internal Validity

Internal validity considers confounding factors that might influence our results. The primary internal threat relates to the 300 seconds per-scenario timeout boundary imposed during the analysis execution. Complex, pathological call graphs risk triggering timeouts before the analysis completes its full traversal. Consequently, the instances we detected and classified represent a lower bound of the actual conflicts present in the codebase. It is possible that the scenarios which timed out contain distributed, complex topologies (such as D3 or G3) that are underrepresented in our final counts.

C. External Validity

External validity addresses the generalizability of our findings. We sourced our merge scenarios exclusively from popular open-source Java projects hosted on GitHub, filtered by build success and the presence of textual conflicts. Consequently, the observed topologies may reflect the specific modularity and encapsulation practices common in open-source Java development, rather than a universal law of software engineering. Crucially, as highlighted in Section IV-B, our dataset strictly comprises scenarios where both developers modified the exact same method (*mutually modified methods*). This design choice inherently constrains the origin of semantic changes to a single file, effectively preventing the manifestation of topologies where conflicting edits originate in distinct files (such as A2, A3, or A4). While this limitation represents a threat to external validity, it is an inherent limitation of the static analysis approach and tools currently available to us. Therefore, the results only reflect layouts that are structurally possible under this constraint. Finally, our findings may not transfer directly to closed-source enterprise projects with different architectural conventions, unmaintained codebases with irregular merge histories, or systems written in languages with different dispatch and scoping mechanisms (e.g., Python or C++).

VII. Related Work

This section positions our quantitative characterization study within the broader context of collaborative development research, highlighting approaches that identify, classify, and mitigate code integration conflicts. We organize the discussion into four categories and explicitly contrast each line of work with our contributions.

A. Empirical Studies on Merge Conflicts

Researchers extensively investigate the challenges of collaborative software development and code integration. Ghiotto et al. [13] investigate the nature of textual merge conflicts across 2,731 open-source Java projects, categorizing the typical syntax clashes developers face daily and quantifying their frequency and resolution complexity. Elias et al. [14] explore the characteristics of semi-structured merge conflicts, analyzing code composition patterns and structural properties of conflicting regions. Campos Junior et al. [15] assess merge tools and strategies, comparing resolution recommendations that assist developers in handling textual integration failures. Cavalcanti et al. [2] evaluate and improve semistructured merge techniques, demonstrating that structure-aware merging reduces the incidence of false conflicts.

The closest empirical study to our work is by Da Silva et al. [6], who investigate the frequency, causes, and resolution patterns of build conflicts in open-source Java projects. While they provide a comprehensive catalogue of build conflict causes, highlighting that missing declarations cause 65% of build conflicts (for example, *Unavailable Symbol*), their focus is strictly on conflicts that break the build process. Similarly, researchers investigate build conflict frequency and early detection. For example, Kasi and Sarma [16] and Brun et al. [3], [17] report on the frequency of build conflicts to motivate proactive conflict minimization and speculative analysis. In an industrial context, Sung et al. [18] explore build conflicts that arise when integrating changes across forks, proposing strategies for understanding and fixing them.

Unlike these studies, which focus on textual, syntactic, or build conflicts that produce visible merge warnings or compilation errors, our work targets dynamic semantic conflicts, behavioral interferences that occur silently even when textual merges and builds succeed without any warnings. We complement the textual and build conflict literature by providing the

first comprehensive catalog of topological layouts for DF, CF, and OA interferences and quantifying their occurrences.

B. Semantic Conflict Detection

Since dynamic semantic conflicts arise from changes in program behavior, researchers traditionally rely on dynamic execution and test generation to detect them. Da Silva et al. [19] propose an automated behavior change detection technique that uses unit test generation to identify semantic conflicts arising from code integration. Brun et al. [3], [17] introduce speculative analysis for proactive conflict detection, executing tentative merges in the background and running project test suites to identify integration failures before they reach the main branch. However, the quality and coverage of the underlying test suite limit test-based methods, which also incur substantial computational costs when run repeatedly. Our study provides the topological layout data that test-based approaches could use to prioritize which structural patterns warrant the most test generation effort.

To overcome the limitations of dynamic execution, other researchers address semantic conflicts through formal methods and static analysis. Horwitz et al. [20] introduce Program Dependency Graphs (PDGs) and System Dependency Graphs (SDGs) to integrate non-interfering program versions, providing semantic guarantees about merge correctness. Sousa et al. [21] propose SafeMerge, a formal verification tool that uses program verification techniques to prove the absence of unwanted merge behaviors. Although theoretically sound, these techniques face scalability challenges on large codebases. Our study operates at a different abstraction level: rather than proposing a new analysis algorithm, we provide a structured catalog of the topological layouts that these tools aim to detect.

The closest works regarding static semantic conflict detection are by Santos de Jesus et al. [4] and Barbosa et al. [5]. Santos de Jesus et al. compare static analyses for improved semantic conflict detection, evaluating DF, CF, and OA analyses across multiple pointer-analysis configurations, while Barbosa et al. investigate the effect of different call-graph construction algorithms on the precision of OA analysis. While they focus on comparing detection accuracy and pointer-analysis sensitivity, our study directly complements their research by systematically formalizing the structural layouts of these conflicts and quantifying how often each specific topology occurs in practice.

C. AI and Machine Learning for Conflict Resolution

Researchers apply Artificial Intelligence (AI) and Machine Learning (ML) to detect and resolve merge conflicts. They leverage deep learning models to resolve textual conflicts. For instance, Svyatkovskiy et al. [22] formulate program merge conflict resolution via neural transformers, treating the problem as a classification task over three-way diffs. Similarly, Dinella et al. [23] propose DeepMerge, a pointer network-based architecture that learns to merge programs by rearranging existing code. Dong et al. [24] further explore this by proposing generative models that can synthesize new tokens to resolve conflicts, rather than merely classifying existing ones. Other studies combine heuristics with Large Language Models (LLMs); Shen et al. [25] use lightweight classifiers for simple strategies and rely on ChatGPT for complex resolution scenarios, while Zhang et al. [26] demonstrate that pre-trained language models can effectively resolve real-world textual and static semantic conflicts. Furthermore, researchers investigate the use of LLMs for detecting semantic conflicts [27] and evaluating their robustness in code generation tasks [28].

While these AI-based techniques show promising results in automating the resolution of textual and build conflicts, our study diverges by focusing on the static detection and characterization of dynamic semantic conflicts. Generative AI approaches still face challenges related to context window limits, hallucinations, and a reliance on compilation errors to locate issues. By contrast, our work contributes a foundational characterization of the structural topologies (DF, CF, and OA) that lead to silent behavioral interferences, which can ultimately inform and improve the prompts, context selection, and training data for future AI-driven resolution tools.

VIII. Conclusion

This paper presented a systematic topological characterization of semantic conflicts in open-source Java projects. We defined a comprehensive catalog of 54 structural layouts that capture how DF (11 layouts), CF (32 layouts), and OA (11 layouts) interferences manifest across file boundaries and call graphs. We then quantified the occurrence of each layout by executing an interprocedural static analysis on 907 real-world merge scenarios from the RefDataset, detecting 32,361 conflict instances.

Our study produced two primary findings. First, semantic conflicts present a stark structural dichotomy: CF conflicts concentrate overwhelmingly in the fully local group which starts and ends in the same file (90.4% of CF instances), while both DF and OA conflicts exhibit highly divergent behavior, with topologies that start in the same file but end in different files accounting for 60.3% and 46.7% of their respective instances. This structural dichotomy demonstrates that different conflict types require fundamentally different analysis strategies: an intraprocedural analysis is sufficient to detect almost all CF interferences, whereas detecting DF and OA conflicts necessitates interprocedural capabilities across different files.

These findings offer actionable guidance for practitioners configuring static analysis tools in daily software integration pipelines, suggesting tiered detection strategies that reserve deeper, multi-file traversal specifically for DF and OA patterns.

Artifact Availability

Our dataset, analysis scripts, topological layout definitions, and raw conflict detection outputs are available in our online appendix to support reproducibility and replication. [11]

REFERENCES

[1] S. Khanna, K. Kunal, and B. C. Pierce, "A formal investigation of diff3," in *Foundations of Software Technology and Theoretical Computer Science*, 2007, pp. 485–496.

[2] G. Cavalcanti, P. Borba, and P. Accioly, "Evaluating and improving semistructured merge," *Proceedings of the ACM on Programming Languages*, vol. 1, no. OOPSLA, pp. 1–27, 2017.

[3] Y. Brun, R. Holmes, M. D. Ernst, and D. Notkin, "Proactive detection of collaboration conflicts," in *Proceedings of the Joint Meeting of the European Software Engineering Conference and the Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE, 2011, pp. 168–178.

[4] G. Santos de Jesus, P. Borba, R. Bonifácio, and M. Barbosa de Oliveira, "Comparing static analyses for improved semantic conflict detection," *Automated Software Engineering*, vol. 33, no. 2, pp. 1–38, 2025.

[5] M. Barbosa de Oliveira, G. Santos de Jesus, P. Borba, and R. Bonifácio, "Evaluating pointer analysis sensitivity for Override Assignment semantic conflict detection," in *Proceedings of the Brazilian Symposium on Software Engineering*, ser. SBES, 2025, pp. 1–10.

[6] L. Da Silva, P. Borba, and A. Pires, "Build conflicts in the wild," *Journal of Software: Evolution and Process*, vol. 34, 2022.

[7] V. Lira, P. Borba, R. Bonifácio, G. Santos, and M. Barbosa, "Reffilter: Improving semantic conflict detection via refactoring-aware static analysis," 2025. [Online]. Available: https://arxiv.org/abs/2510.01960

[8] R. Vallée-Rai, P. Co, E. Gagnon, L. Hendren, P. Lam, and V. Sundaresan, "Soot: A java bytecode optimization framework," in *CASCON First Decade High Impact Papers*, 2010, pp. 214–224.

[9] O. Lhoták and L. Hendren, "Scaling java points-to analysis using spark," in *International Conference on Compiler Construction*, 2003, pp. 153–169.

[10] Anonymous, "The impact of static analysis depth limit on semantic conflict detection," in *Anonymous Proceedings*, 2026.

[11] ——, "Online appendix," https://anonymous.4open.science/r/SCRAM-appendix, 2026, accessed: 2026.

[12] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012.

[13] G. Ghiotto, L. Murta, M. Barberán, and A. van der Hoek, "On the nature of merge conflicts: a study of 2,731 open source Java projects hosted by GitHub," in *Proceedings of the International Conference on Software Engineering*, ser. ICSE, 2018, pp. 1–14.

[14] G. Elias, G. Cavalcanti, P. Borba, and P. Accioly, "Understanding semistructured merge conflict characteristics in open-source Java projects," in *Proceedings of the Brazilian Symposium on Software Engineering*, ser. SBES, 2020, pp. 1–10.

[15] L. Campos Junior, G. Cavalcanti, P. Borba, and P. Accioly, "An empirical assessment of merge tools and strategies for collaborative software development," in *Proceedings of the International Conference on Software Engineering*, ser. ICSE, 2023, pp. 1–12.

[16] B. K. Kasi and A. Sarma, "Cassandra: Proactive conflict minimization through optimized task scheduling," in *International Conference on Software Engineering*, 2013, pp. 732–741.

[17] Y. Brun, R. Holmes, M. D. Ernst, and D. Notkin, "Early detection of collaboration conflicts and risks," *IEEE Transactions on Software Engineering*, vol. 39, no. 10, pp. 1358–1375, 2013.

[18] C. Sung, S. K. Lahiri, M. Kaufman, P. Choudhury, and C. Wang, "Towards understanding and fixing upstream merge induced conflicts in divergent forks: An industrial case study," in *International Conference on Software Engineering*, 2020, pp. 172–181.

[19] L. Da Silva, P. Borba, G. Cavalcanti, and P. Accioly, "Detecting semantic conflicts via automated behavior change detection," in *Proceedings of the International Conference on Software Engineering*, ser. ICSE, 2020, pp. 1–12.

[20] S. Horwitz, J. Prins, and T. Reps, "Integrating noninterfering versions of programs," *ACM Transactions on Programming Languages and Systems*, vol. 11, no. 3, pp. 345–387, 1989.

[21] M. Sousa, I. Dillig, and S. K. Lahiri, "Verified three-way program merge," in *Proceedings of the Conference on Object-Oriented Programming, Systems, Languages, and Applications*, ser. OOPSLA, 2018, pp. 1–29.

[22] A. Svyatkovskiy, S. Fakhoury, N. Ghorbani, T. Mytkowicz, E. Dinella, C. Bird, J. Jang, N. Sundaresan, and S. K. Lahiri, "Program merge conflict resolution via neural transformers," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2022, p. 822–833. [Online]. Available: https://doi.org/10.1145/3540250.3549163

[23] E. Dinella, T. Mytkowicz, A. Svyatkovskiy, C. Bird, M. Naik, and S. Lahiri, "Deepmerge: Learning to merge programs," *IEEE Trans. Softw. Eng.*, p. 1599–1614, 2023. [Online]. Available: https://doi.org/10.1109/TSE.2022.3183955

[24] J. Dong, Q. Zhu, Z. Sun, Y. Lou, and D. Hao, "Merge conflict resolution: Classification or generation?" in *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*, 2024, p. 1652–1663. [Online]. Available: https://doi.org/10.1109/ASE56229.2023.00155

[25] C. Shen, W. Yang, M. Pan, and Y. Zhou, "Git merge conflict resolution leveraging strategy classification and llm," in *2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security (QRS)*, 2023, pp. 228–239.

[26] J. Zhang, T. Mytkowicz, M. Kaufman, R. Piskac, and S. K. Lahiri, "Using pre-trained language models to resolve textual and semantic merge conflicts (experience paper)," in *International Symposium on Software Testing and Analysis*, 2022, pp. 77–88. [Online]. Available: https://dl.acm.org/doi/abs/10.1145/3533767.3534396

[27] J. Chun, G. Zhang, and M. Xia, "Conflictlens: Llm-based conflict resolution training in romantic relationship," in *Adjunct Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology*, 2025. [Online]. Available: https://doi.org/10.1145/3746058.3758422

[28] J. Liu, S. Xu, Y. Yao, Y. Shen, Y. Zhao, X. Zhu, P. Yu, F. Xu, and X. Ma, "Robustness evaluation and enhancement of llms in code generation: an empirical study," *Empirical Softw. Engg.*, 2026. [Online]. Available: https://doi.org/10.1007/s10664-026-10856-w